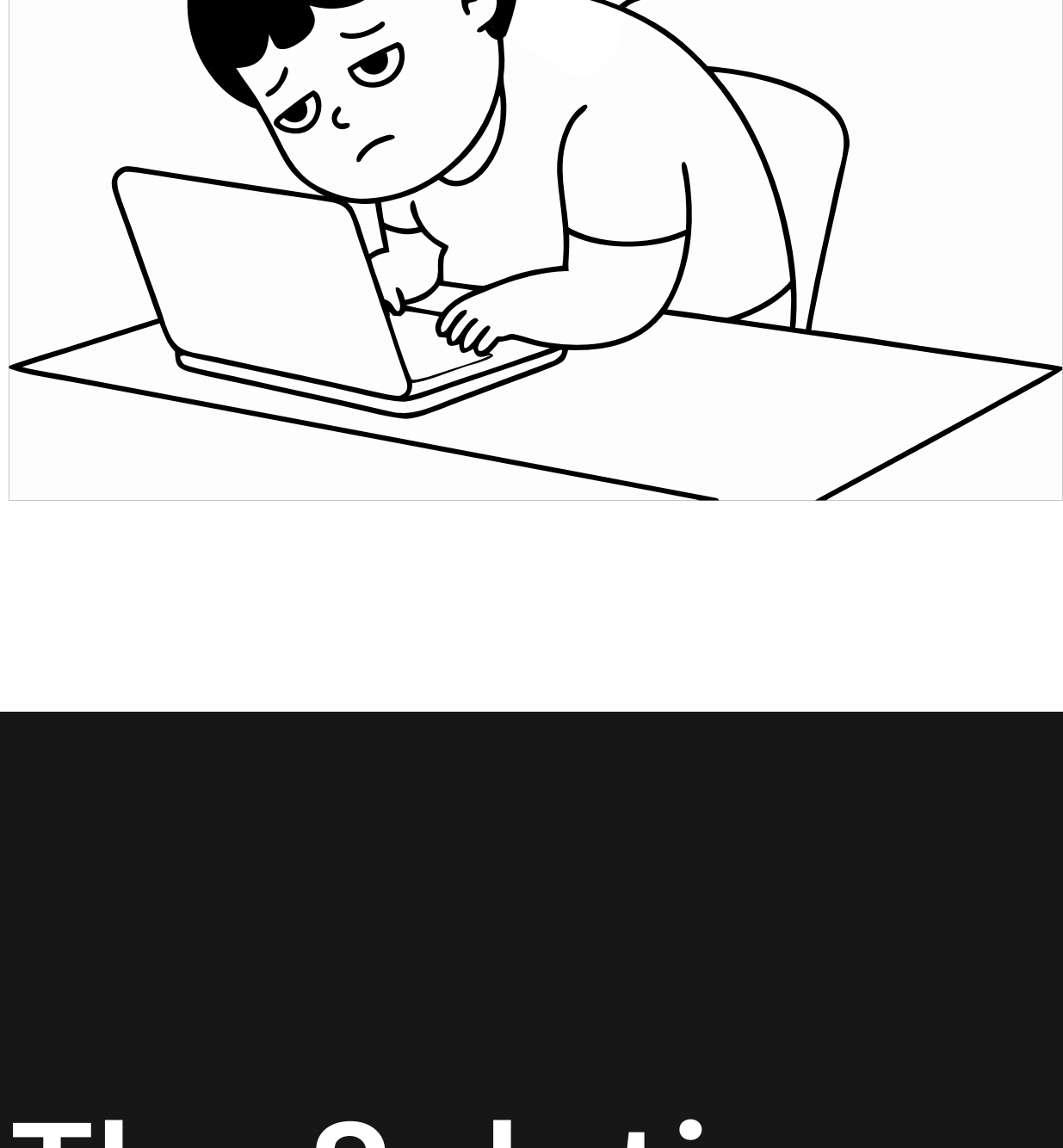


Nexus

Self-Serve Feature Provisioning & RBAC Governance

B2B SaaS Role: Design Intern Timeline: 7 weeks

Designing a self-serve configuration engine to eliminate developer dependencies, streamline role mapping, and protect production access controls at scale.

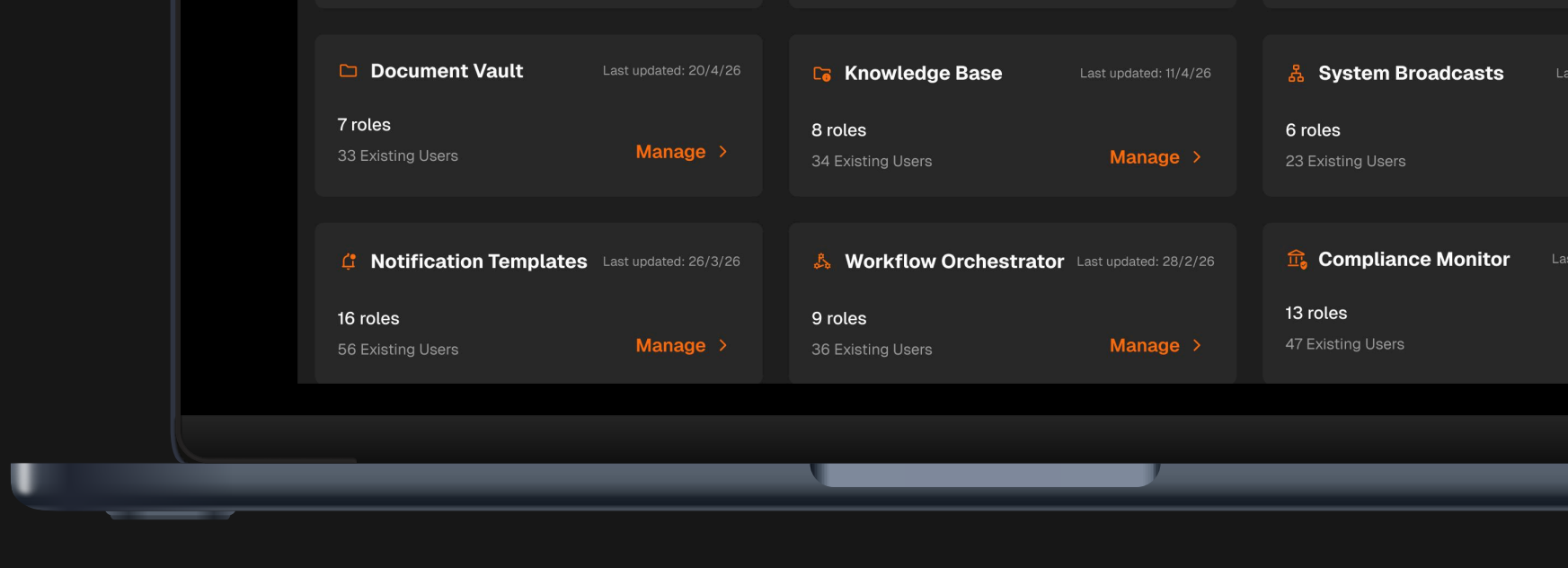


The Problem

Every time we onboarded a feature, mapped a RBAC role, or removed an old one, a developer had to make the change directly in the backend. Routine admin work was constantly queued behind engineering sprint priorities. It was slow, tedious, and did not scale.

The Solution

We built a centralized self-serve flow inside Admin Console so operations could handle these updates directly. Admins can now configure features and map roles through a single interface, removing developer involvement entirely for routine admin changes.



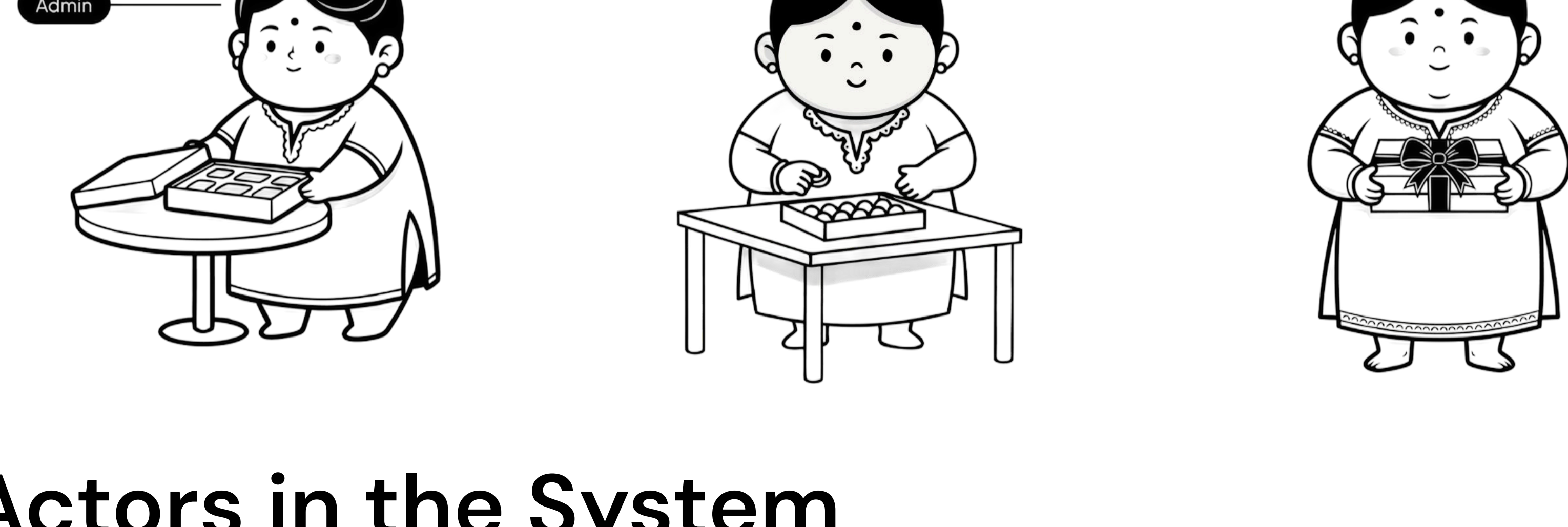
The Workflow

The conceptual mental model

Admin creates a **Feature**

Assigns the **Roles**

Deploys changes **Live**



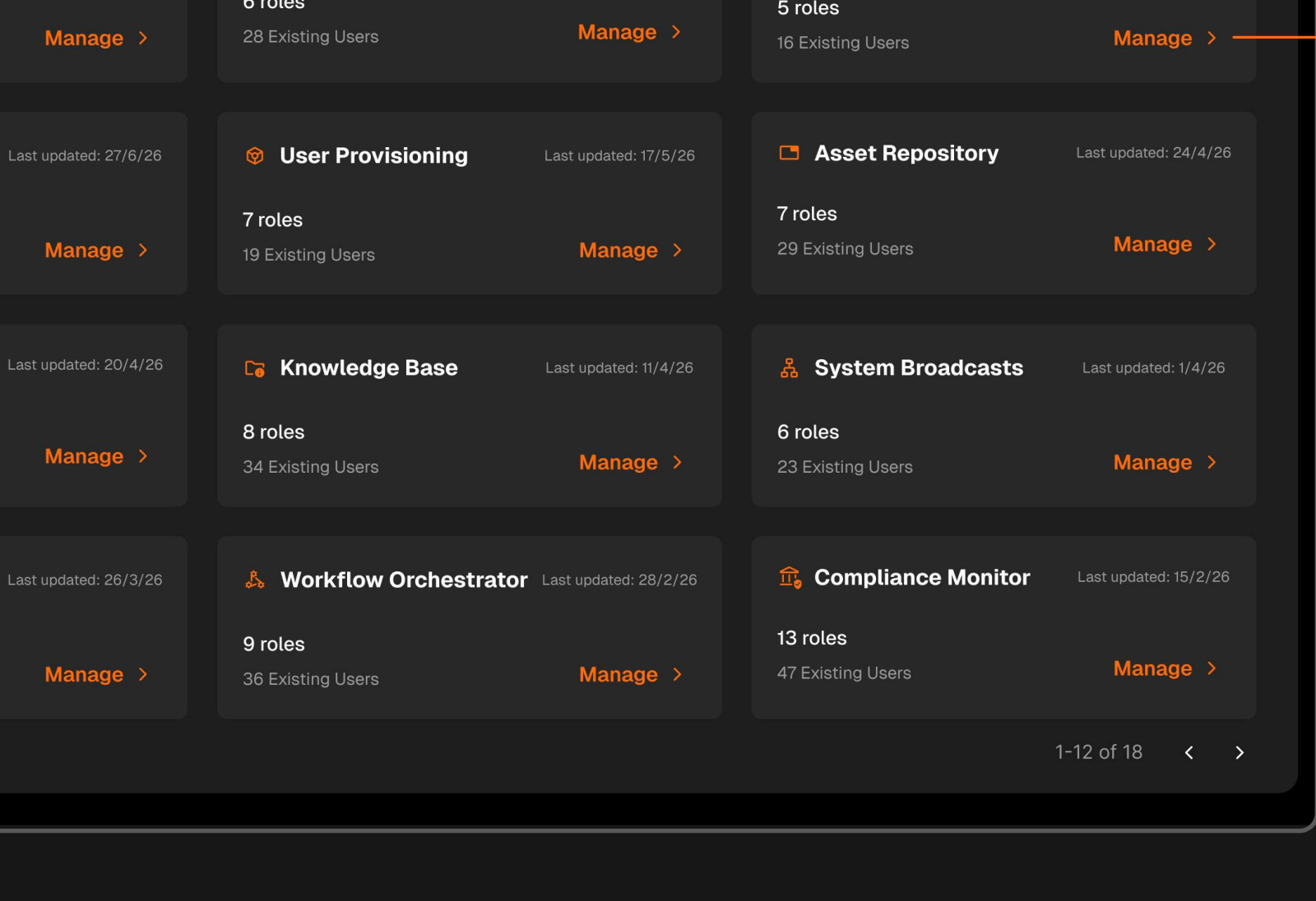
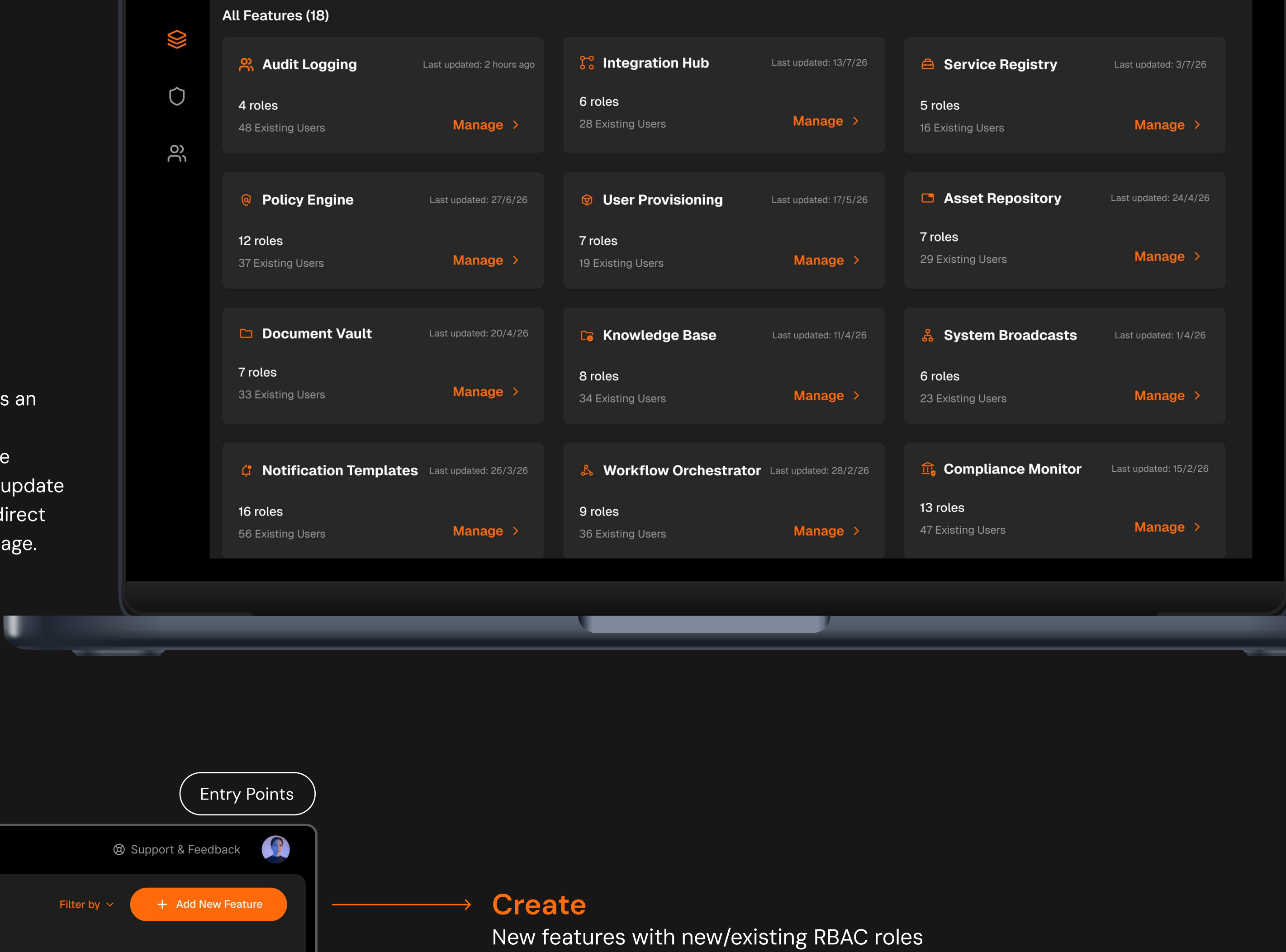
Actors in the System

PRIMARY USER
Non-technical platform operators
Onboard new features and assign access permissions quickly
Blocked by engineering sprint queues just to make routine database updates.

BOTTLENECKED TEAM
Core developers maintaining the backend
Ship customer-facing roadmap features.
Constantly interrupted by repetitive, manual configuration tickets.

Feature Directory

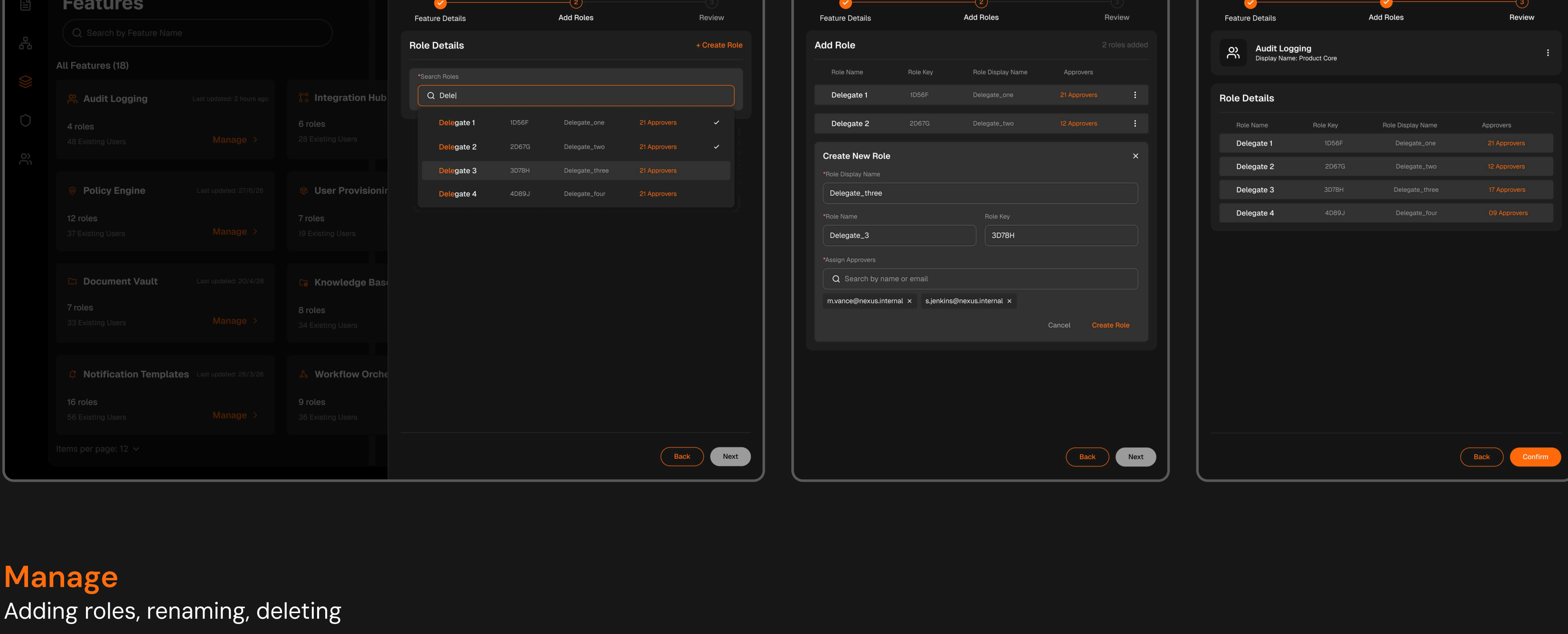
A real-time catalog that provides an instant inventory of all platform services, displaying assigned role density, active user counts, and update recency for every feature, with direct entry points to provision or manage.



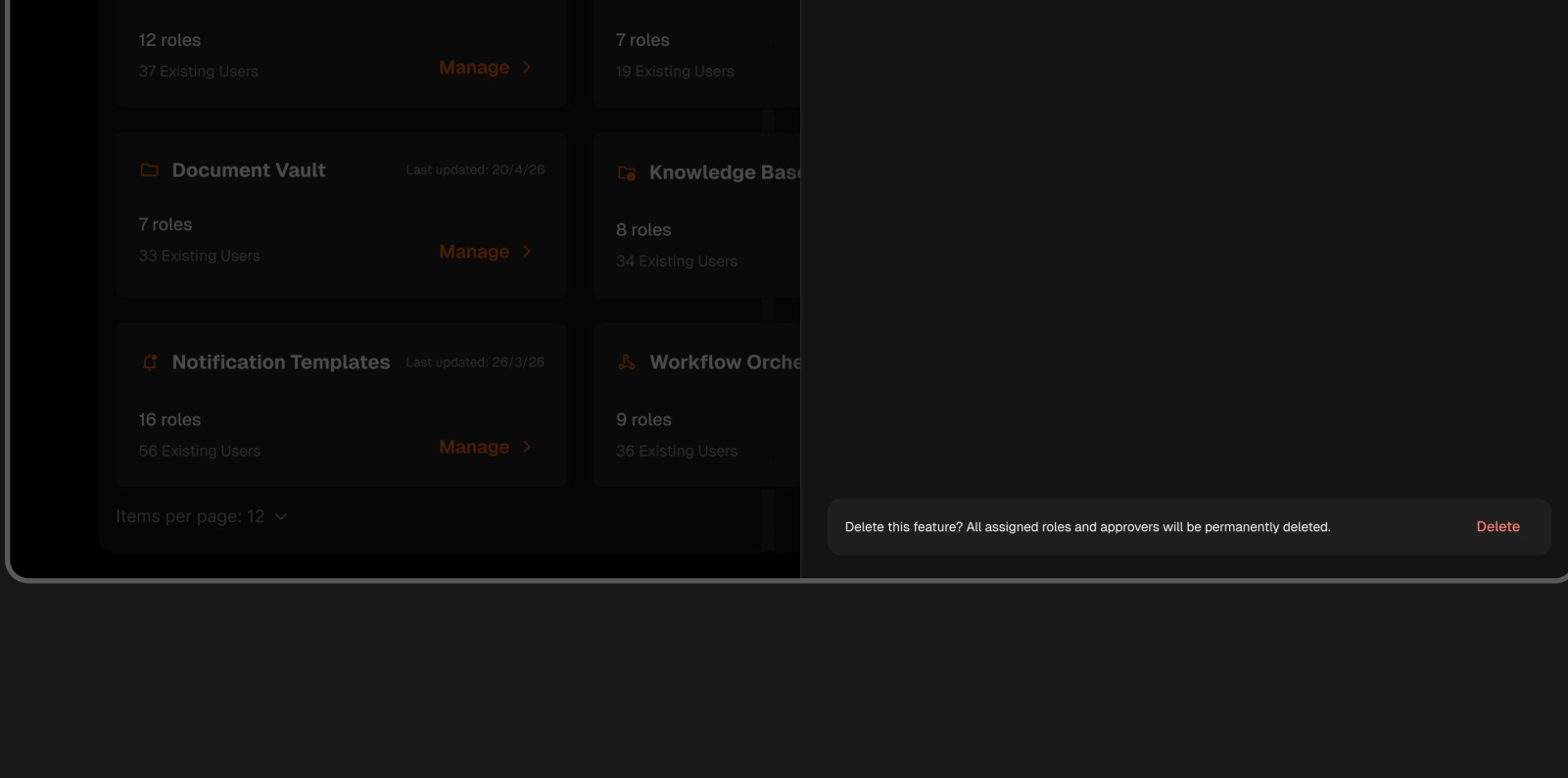
Create
New features with new/existing RBAC roles

Manage
Adding roles, renaming, deleting

Create
New features with new/existing RBAC roles



Manage
Adding roles, renaming, deleting



Key Design Decisions

A long form added scroll, became difficult to navigate & overwhelming as more roles were created.

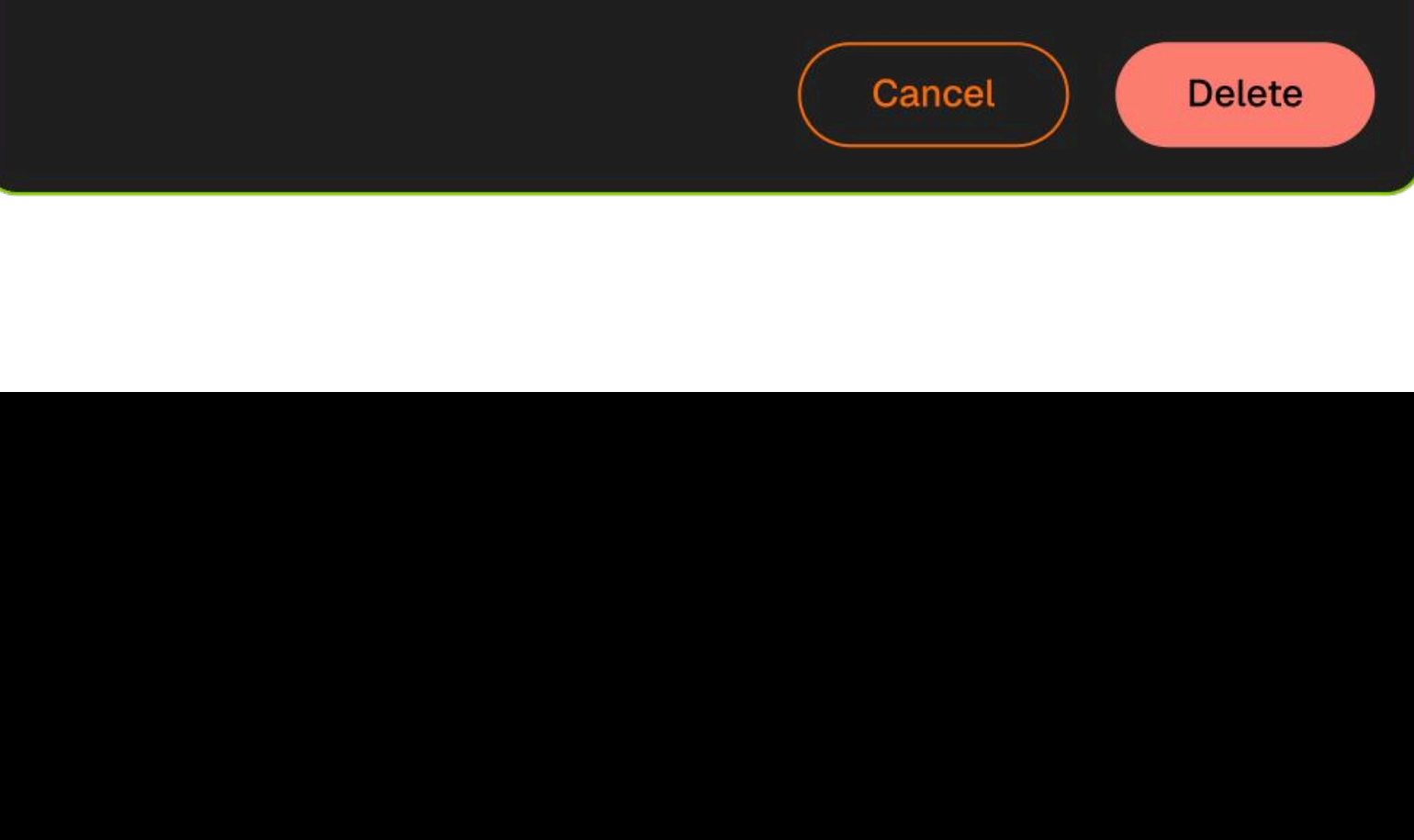
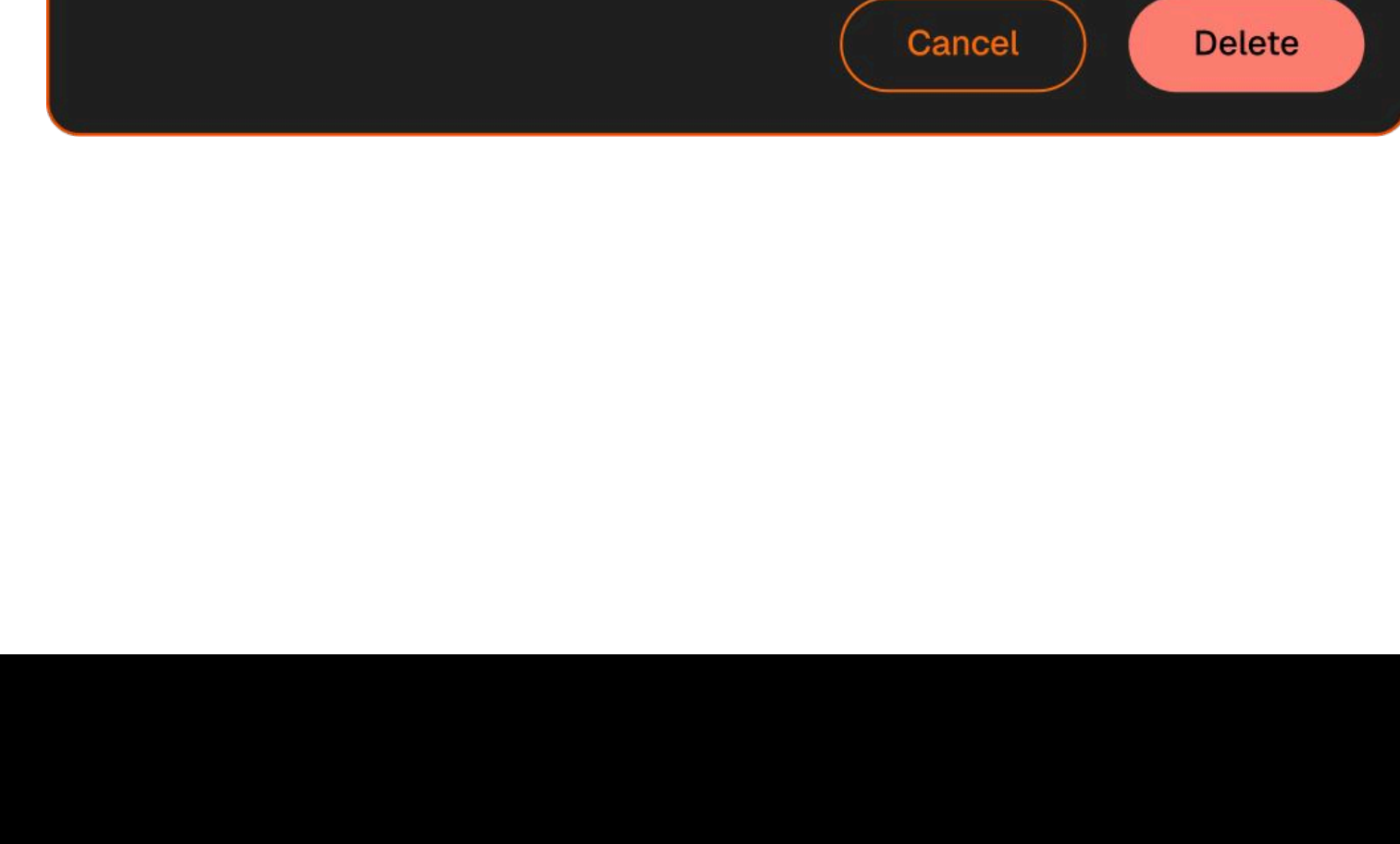
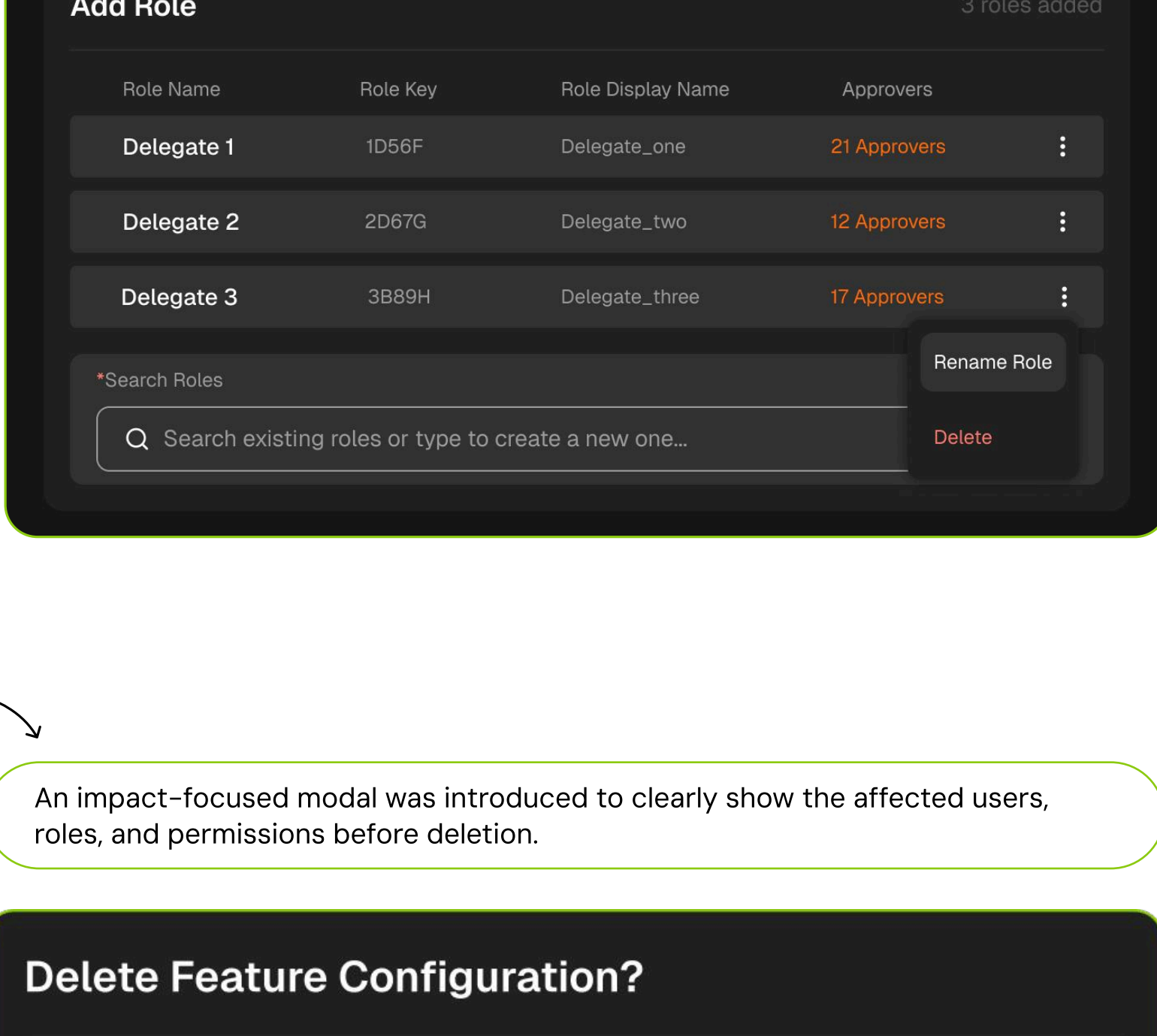
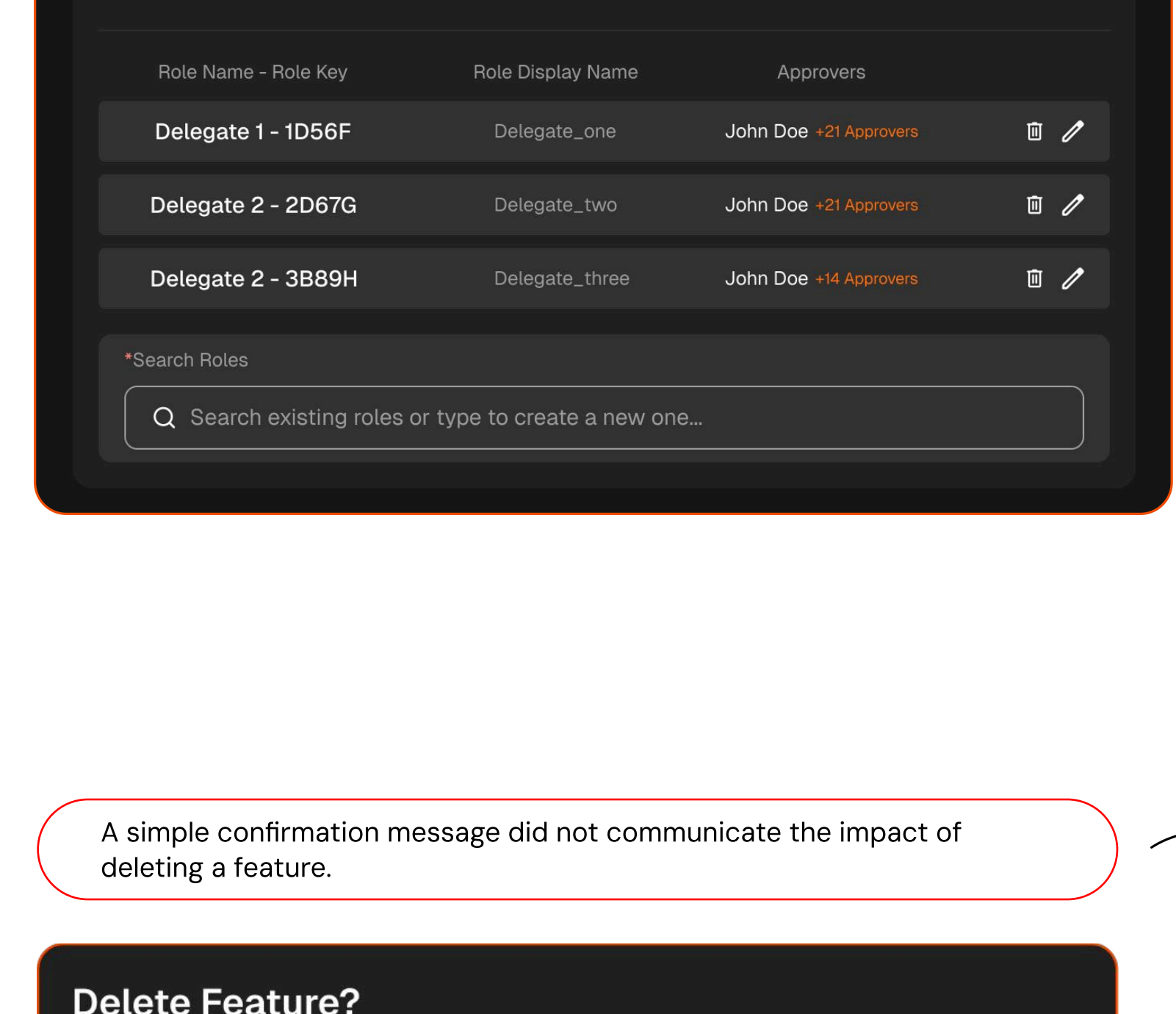
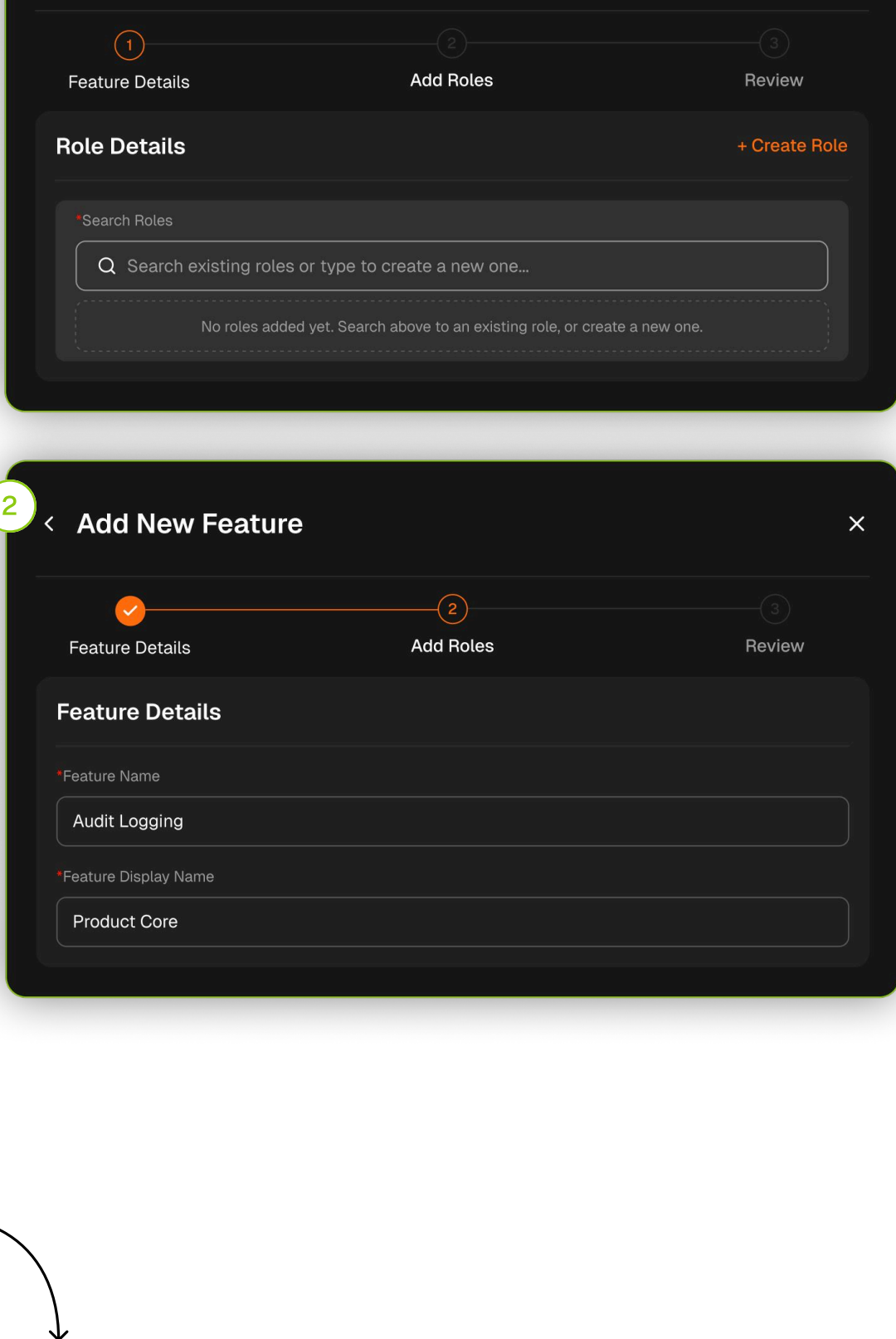
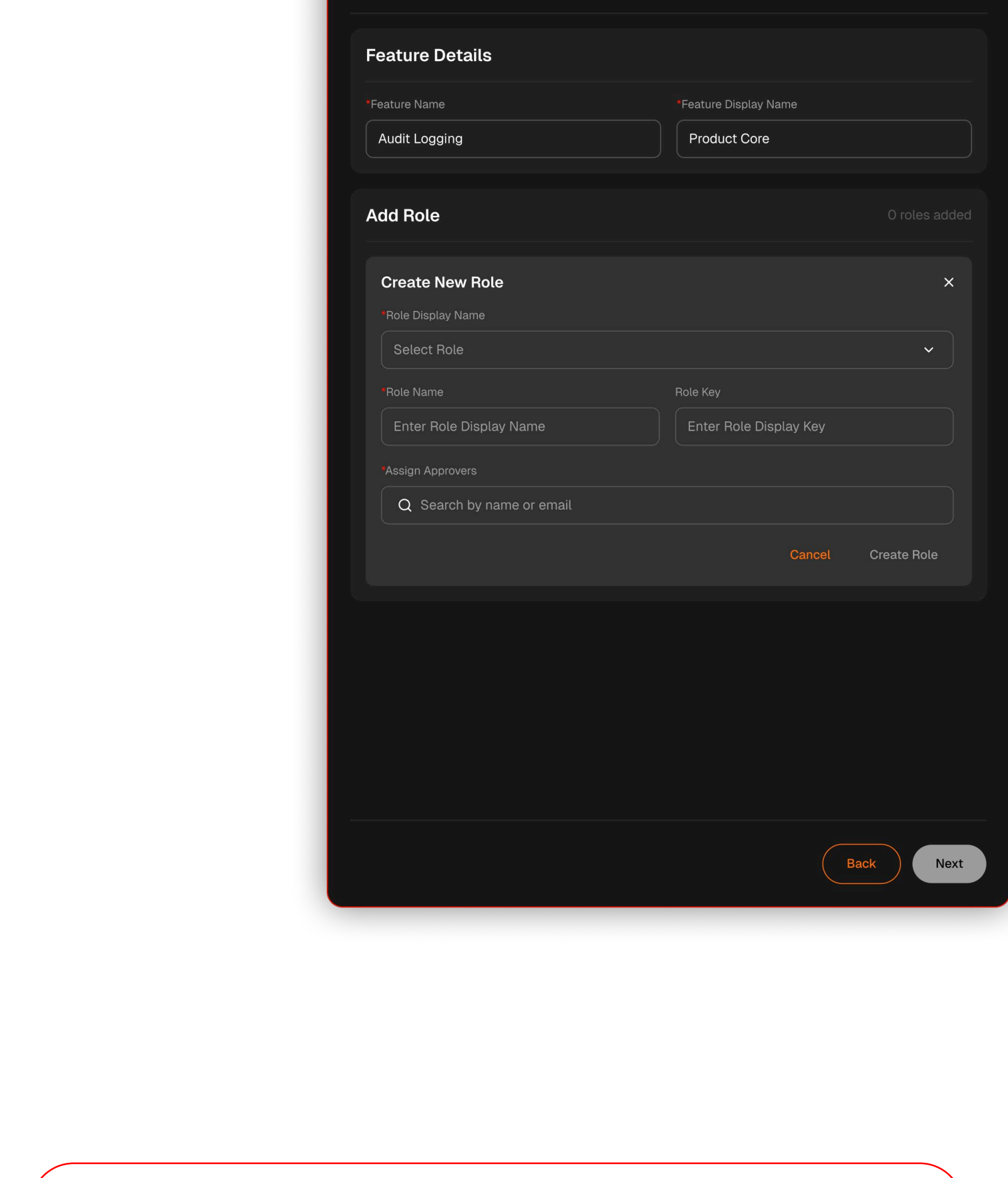
Split the intake into a 3-step wizard to separate mental tasks and enforce a final review step

Delete & rename on every row added visual clutter & increased the chance of accidental deletion.

Deletion was placed inside a 3-dot kebab menu, keeping the table clean while making deletion more intentional.

A simple confirmation message did not communicate the impact of deleting a feature.

An impact-focused modal was introduced to clearly show the affected users, roles, and permissions before deletion.



Retrospective

Defending the UX with Intent

Standing up for user needs during handoff by defending the logic behind critical safeguards, while staying grounded in engineering feasibility.



Reaching mutual decisions with POs

Understanding the business pressures POs face helped us make mutual decisions that hit business goals without hurting the user experience.